Timed Scenario Expressions and Realisability

Neda Saeedloei* and Feliks Kluźniak

* Towson University
nsaeedloei@towson.edu

Abstract. We introduce expressions formed from distributed timed scenario. Such expressions allow us to specify complex distributed behaviours in a hierarchical fashion. We define the semantics and consistency of a scenario expression. We then address the problem of realisability of sets of sequential timed scenarios as scenario expressions and present an efficient algorithm which produces a non-trivial scenario expression that preserves the semantics of the original set of scenarios. Our method directly handles explicit time constraints and produces a scenario expression whose time constraints are inferred from those of the realised set.

1 Introduction

The problem of realisability of global models, e.g., global (session) types [1–4], global choreographies [5], global labelled transition systems [6] and global languages has attracted many researchers over the past several years. A realisation of a global model is a set of collaborating computing entities which are executed concurrently, and whose behaviours, obtained from their (synchronous or asynchronous) interactions, are exactly those that are obtained from the global models. For example, a global model can be the parallel composition of a set of labelled transition systems, each of which models the local behaviour of a component. An interesting, but challenging question is how to determine whether a global model is realisable and, if so, how to obtain a realisation.

We are interested in the problem of realisability for global models that take the form of sets of sequential timed scenarios. Given such a set, our goal is to obtain a *timed scenario expression* that realises the set.

In earlier work we developed the notions of sequential timed scenarios (scenarios for short) [7] and distributed timed scenarios [8]. A scenario is a sequence of (names of) events along with a set of constraints on the times at which these events occur (see Sec. 2.1 for more details). A distributed timed scenario (DTS) is a collection of components, each of which is a timed scenario (see Sec. 2.2 for more details). Each component performs its specified actions, represented as a sequence of events, concurrently with other components. Components can communicate or synchronise with each other by performing the same action simultaneously. A DTS, Ξ, is viewed as a parallel composition of its components and its semantics is defined as a set of distributed behaviours, i.e., the set of all possible interleavings of events in the components of Ξ that satisfy the constraints and the synchronising requirements of Ξ.

Given a set, $\mathcal{S}$, of scenarios over a set of events, $\Sigma_{\mathcal{S}}$, the stable distance relation [8] of $\mathcal{S}$ is $\mathcal{DR}_{\mathcal{S}} = (\prec_{\mathcal{S}}, \mathcal{DF}_{\mathcal{S}})$. The partial order $\prec_{\mathcal{S}}$ captures all the ordering dependencies between events in $\mathcal{S}$, while the distance function, $\mathcal{DF}_{\mathcal{S}}$, expresses constraints on the time between every pair of events e and e', where $e \prec_{\mathcal{S}} e'$. A projection of $\mathcal{DR}_{\mathcal{S}}$ is a scenario over $\Sigma_{\mathcal{S}}$ whose sequence of events agrees with $\prec_{\mathcal{S}}$ and whose constraints satisfy the constraints captured by $\mathcal{DF}_{\mathcal{S}}$.

In the current paper we introduce *timed scenario expressions* and solve the problem of realisability of sets of scenarios as scenario expressions. So our contribution is twofold:

First, we introduce a limited form of timed scenario expressions (expressions for short), as a means of constructing hierarchical models of complex systems, whose constituent parts are distributed systems (Sec. 3). Intuitively, an expression is built by repeatedly applying three operators, namely choice, concatenation and parallel composition (denoted by $|$, $\circ$ and $||$[1], respectively), to distributed scenarios. In its simplest form an expression is just a single DTS. Given an expression E, we define its semantics, denoted by $[\![E]\!]$, in terms of the set of distributed behaviours that are expressed by E.

Second, we tackle the problem of realisability of sets of (sequential) scenarios as expressions (Sec. 4). Given a finite set, $\mathcal{S}$, of scenarios, we produce a scenario expression, E, that realises $\mathcal{S}$, i.e., $[\![E]\!]$ is identical to the union of the sets of behaviours specified by the members of $\mathcal{S}$.

We identify two criteria for realisability of $\mathcal{S}$ as a single DTS: $\mathcal{S}$ is realisable by a DTS iff (i) it is *order-closed*, and (ii) all its members are *compatible*, in terms of their constraints, with $\mathcal{DR}_{\mathcal{S}}$. Intuitively, $\mathcal{S}$ is order-closed if, for every projection of $\mathcal{DR}_{\mathcal{S}}$, there is a scenario in $\mathcal{S}$ with that exact sequence of events, and vice versa. A scenario $sc \in \mathcal{S}$ is compatible with $\mathcal{DR}_{\mathcal{S}}$ if its set of constraints is equivalent to the set of constraints of the projection of $\mathcal{DR}_{\mathcal{S}}$ that has the same sequence of events as sc.

If $\mathcal{S}$ is not realisable by a single DTS, then it must be divided into partitions, in such a way that each partition is order-closed and all its members are compatible with the distance relation of the partition. Once every partition is realisable by a single DTS, the resulting expression will be a choice between these distributed scenarios.

The novelty of our work is primarily in the use of (stable) distance relations [8] as a linchpin of an efficient method for the realisation problem. By identifying the necessary and sufficient conditions for the realisability of a set of scenarios as a DTS, we address the problem that was left open in our earlier work [8].

Related Work In our earlier work [8] we developed an algorithm for solving the problem of realisability of sets of (sequential) scenarios as distributed scenarios. Given a set of scenarios, our algorithm determines whether the set is realisable by a *single* DTS, i.e., whether there is a DTS whose semantics is identical to the union of the behaviours described by the members of the set. If so, the algorithm

[1] We do not consider the Kleene star ($*$) operator here, as its introduction opens up interesting questions that cannot be addressed in the confines of the current paper.

produces such a DTS. Despite some similarities, the problem that we tackle in the current paper is fundamentally different.

The comparison of our sequential timed scenarios with simple temporal networks (STNs) [9], as well as with message sequence charts (MSCs) [10, 11], is presented elsewhere [8]. Similarities between the distributed behaviours of distributed timed scenarios and the timed traces of Balaguer et al. [12] are also discussed in our earlier work [8].

The problem of realisability has been investigated in a variety of settings, e.g., for message sequence charts [13], global session types [1–4], pomsets [14], global labelled transition systems [6], choreography languages [5] and team automata [15]. Unlike in all these works the realisability of our sets of sequential scenarios as scenario expressions is determined by precise time analysis enabled by our distance relations. That is, in our setting we are able to directly reason about realisability of models that include explicit time constraints and produce realisations that involve time constraints.

2 Preliminaries

This section briefly recounts our earlier work [7, 8].

2.1 Sequential Timed Scenarios

Let Σ be a finite set of symbols called *events*. A *behaviour*[2] over Σ is a non-empty sequence $(e_0, t_0)(e_1, t_1)(e_2, t_2)\dots$, such that $e_i \in \Sigma$, $t_i \in \mathbb{R}^{\geq 0}$ and $t_{i-1} \leq t_i$ for $i \in \{1, 2\dots\}$. For a finite behaviour $\mathcal{B} = (e_0, t_0)(e_1, t_1)\dots(e_{n-1}, t_{n-1})$ of length n ($n \in \mathbb{N}^{>0}$), and for any $0 \leq i < j < n$, the *distance*, in time units, of event j from event i in $\mathcal{B}$ is denoted by $t^{\mathcal{B}}_{ij}$. That is, $t^{\mathcal{B}}_{ij} = t_j - t_i$.

A *sequential timed scenario* (*scenario* for short) of length $n \in \mathbb{N}$ over Σ is a pair $(\mathcal{E}, \mathcal{C})$, where $\mathcal{E} = e_o e_1 ... e_{n-1}$ is a non-empty sequence of events from Σ, and $\mathcal{C} \subset \Phi(n)$ is a finite set of constraints. A constraint in $\Phi(n)$ is of the form $b \sim a$, where b is the symbol $\tau_{i,j}$ (for some integers $0 \leq i < j < n$), $\sim \in \{\leq, \geq, =\}$[3] and $a \in \mathbb{Q}$ is a constant. The interpretation is that $\tau_{i,j}$ is the time distance between the i-th and the j-th events in the behaviours described by a scenario. The constraints $\tau_{i,j} \geq 0$ and $\tau_{i,j} \leq \infty$ are called *default constraints*.

We use $\Sigma_\xi = \{e_0, e_1, \dots, e_n\} \subseteq \Sigma$ to denote the set of events in $\mathcal{E}$.

A behaviour $\mathcal{B} = (e_0, t_0)(e_1, t_1)\dots(e_{n-1}, t_{n-1})$ over Σ is *allowed* by scenario $\xi = (\mathcal{E}, \mathcal{C})$ iff $\mathcal{E} = e_0 \dots e_{n-1}$ and every $\tau_{i,j} \sim a$ in $\mathcal{C}$ evaluates to true after $\tau_{i,j}$ is replaced by $t^{\mathcal{B}}_{ij}$. The *semantics* of scenario ξ, denoted by $[\![\xi]\!]$, is the set of behaviours that are allowed by ξ. A scenario ξ is *consistent* iff $[\![\xi]\!] \neq \emptyset$.

For example, $\xi = (abc, \{\tau_{0,1} \geq 3, \tau_{0,2} \leq 7\})$ is a consistent scenario: $[\![\xi]\!] = \{(a, t_0)(b, t_1)(c, t_2) \mid t_0 \leq t_1 \leq t_2 \wedge t_1 - t_0 \geq 3 \wedge t_2 - t_0 \leq 7\}$.

[2] The notion of "behaviour" is equivalent to that of Alur's "timed word" [16].

[3] To keep the presentation compact, we do not allow sharp inequalities. They are mostly of theoretical interest: in practice we can measure time only with some finite granularity γ, so $x < c$ is for all practical purposes equivalent to $x \leq c - \gamma$.

2.2 Distributed Timed Scenarios

The scenario $\xi = (\mathcal{E}, \mathcal{C})$ is in *normalised form* iff every event of $\mathcal{E}$ has only one occurrence in $\mathcal{E}$[4]. If ξ is in normalised form, then each event is uniquely identified by its name. If event i in ξ is a, we will use t_a (instead of t_i) to refer to the time of a in a behaviour $\mathcal{B} \in [\![\xi]\!]$. If event j $(i < j)$ in ξ is b, we will use $\tau_{a,b}$ (instead of $\tau_{i,j}$) in a constraint that restricts the time distance between a and b, and $t^{\mathcal{B}}_{ab}$ (instead of $t^{\mathcal{B}}_{ij}$) to represent the time distance between a and b in $\mathcal{B}$. Naturally, the sequence $\mathcal{E} = e_o e_1 ... e_n$ of a normalised scenario ξ induces a total order on Σ_ξ. We use $<_\xi$ to denote this total order: for two events e_i and e_j in $\mathcal{E}$, $e_i <_\xi e_j \iff i < j$.

A *distributed timed scenario* (*DTS* or *distributed scenario* for short) over Σ is a finite collection of (sequential) scenarios, denoted by $\Xi = \mathcal{D}(\xi_1, \ldots, \xi_n)$, such that each ξ_j $(1 \leq j \leq n)$ is consistent and is in normalised form. We refer to each ξ_j as a *component of* Ξ. We use $\Sigma_\Xi = \Sigma_{\xi_1} \cup \cdots \cup \Sigma_{\xi_n} \subseteq \Sigma$ to denote the set of events of Ξ. We say Ξ is *well-formed* iff $\prec_\Xi \subseteq \Sigma_\Xi \times \Sigma_\Xi$, defined as the transitive closure of $\bigcup_{\xi \in \Xi} <_\xi$, is a strict partial order on Σ_Ξ.

For $\mathcal{B}$, a behaviour over Σ, and ξ, a scenario, such that $\Sigma_\xi \subseteq \Sigma$, the *restriction* of $\mathcal{B}$ to ξ, denoted by $\mathcal{B}_\xi$, is the maximal subsequence of $\mathcal{B}$ such that every event of $\mathcal{B}_\xi$ is in Σ_ξ. If $\Xi = \mathcal{D}(\xi_1, \ldots, \xi_n)$ is a DTS over Σ, then behaviour $\mathcal{B}$ over Σ is *allowed* by Ξ iff, for every component ξ_i of Ξ $(1 \leq i \leq n)$, $\mathcal{B}_{\xi_i} \in [\![\xi_i]\!]$. Clearly, if Ξ is not well-formed, then it allows no behaviours. The *semantics* of Ξ, denoted by $[\![\Xi]\!]$, is the set of behaviours that are allowed by Ξ. A DTS Ξ is *consistent* iff $[\![\Xi]\!] \neq \emptyset$. For a DTS Ξ, and $e \in \Sigma_\Xi$ if $|\{\xi \in \Xi \mid e \in \Sigma_\xi\}| > 1$, then e is called a *synchronising event* for Ξ[5].

For example, $\Xi_1 = \mathcal{D}(\xi_1, \xi_2)$, where $\xi_1 = (habc, \{\tau_{h,a} \leq 3, \tau_{a,b} \leq 4\})$ and $\xi_2 = (hefbd, \{\tau_{h,e} \geq 4\})$ is a consistent DTS: behaviour $\mathcal{B} = (h,0)(a,3)(e,5)(f,6)(b,7)(c,9)(d,10)$ is in $[\![\Xi_1]\!]$. This is because $\mathcal{B}_{\xi_1} = (h,0)(a,3)(b,7)(c,9)$ is in $[\![\xi_1]\!]$ and $\mathcal{B}_{\xi_2} = (h,0)(e,5)(f,6)(b,7)(d,10)$ is in $[\![\xi_2]\!]$. Notice that $\mathcal{B}$ consists of an interleaving of the timed events of ξ_1 and ξ_2, except that the events h and b occur only once. These events, which occur simultaneously in ξ_1 and ξ_2, are the synchronising events of Ξ_1. One can think of the behaviours of the components of Ξ_1 as taking place concurrently, but synchronising on the shared events ("rendezvous" synchronisation). In other words, Ξ_1 can be viewed as a *parallel composition* of its components. If $\Xi = \mathcal{D}(\xi_1, \ldots, \xi_n)$, then we often write $\Xi = \xi_1 \| \ldots \| \xi_n$.

Distance relations If $\Xi = \mathcal{D}(\xi_1, \ldots, \xi_n)$, where $\xi_i = (\mathcal{E}_i, \mathcal{C}_i)$ $(1 \leq i \leq n)$, then let $\mathcal{C}(\Xi) = \bigcup_{1 \leq i \leq n} \mathcal{C}_i$. If Ξ is a well-formed DTS, for $(e_1, e_2) \in \prec_\Xi$, we define $L_{e_1 e_2} = \{l \mid \tau_{e_1,e_2} \geq l \in \mathcal{C}(\Xi)\}$ and $H_{e_1 e_2} = \{h \mid \tau_{e_1,e_2} \leq h \in \mathcal{C}(\Xi)\}$. The distance relation of Ξ is $\mathcal{DR}_\Xi = (\prec_\Xi, \mathcal{DF}_\Xi)$, where $\mathcal{DF}_\Xi : \prec_\Xi \mapsto \mathbb{Q} \times \mathbb{Q}$ is a total function and $\mathcal{DF}_\Xi(e_1, e_2) = [l_{e_1 e_2}, h_{e_1 e_2}]$, such that (i) if $L_{e_1 e_2} \neq \emptyset$, then $l_{e_1 e_2} = \max(L_{e_1 e_2})$, otherwise $l_{e_1 e_2} = 0$ and (ii) if $H_{e_1 e_2} \neq \emptyset$, then $h_{e_1 e_2} = \min(H_{e_1 e_2})$,

[4] If there is more than one event with the same name, say a, in a scenario of length n, the kth $(1 < k \leq n-1)$ occurrence of a can be replaced with a_k.

[5] We assume that any renaming of the synchronising events necessitated by normalising the components of a DTS is performed consistently across all the components. This does not pose any practical limitations.

$$
\begin{array}{l}
l_{e_1e_2} + l_{e_2e_3} > l_{e_1e_3} \longrightarrow l_{e_1e_3} := l_{e_1e_2} + l_{e_2e_3} \\
l_{e_1e_2} + h_{e_2e_3} > h_{e_1e_3} \longrightarrow h_{e_2e_3} := h_{e_1e_3} - l_{e_1e_2} \\
l_{e_1e_3} > l_{e_1e_2} + h_{e_2e_3} \longrightarrow l_{e_1e_2} := l_{e_1e_3} - h_{e_2e_3} \\
h_{e_1e_2} + l_{e_2e_3} > h_{e_1e_3} \longrightarrow h_{e_1e_2} := h_{e_1e_3} - l_{e_2e_3} \\
l_{e_1e_3} > h_{e_1e_2} + l_{e_2e_3} \longrightarrow l_{e_2e_3} := l_{e_1e_3} - h_{e_1e_2} \\
h_{e_1e_3} > h_{e_1e_2} + h_{e_2e_3} \longrightarrow h_{e_1e_3} := h_{e_1e_2} + h_{e_2e_3}
\end{array}
$$

Fig. 1: Rules for stabilising a distance relation

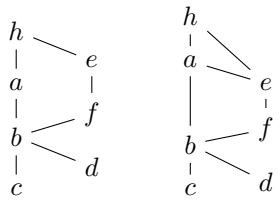

Fig. 2: $\prec_{\Xi_1}$ and $\prec^x_{\Xi_1}$

otherwise $h_{e_1e_2} = \infty$. $\mathcal{DR}_\Xi$ is *stable* iff for every $e_1 \prec_\Xi e_2$, $l_{e_1e_2} \leq h_{e_1e_2}$, and for every $e_1 \prec_\Xi e_2 \prec_\Xi e_3$, the following inequalities hold

$$l_{e_1e_2} + l_{e_2e_3} \leq l_{e_1e_3} \leq \left\{ \begin{array}{l} l_{e_1e_2} + h_{e_2e_3} \\ h_{e_1e_2} + l_{e_2e_3} \end{array} \right\} \leq h_{e_1e_3} \leq h_{e_1e_2} + l_{e_2e_3} \tag{1}$$

If Ξ is consistent, $\mathcal{DR}_\Xi$ can be stabilised by iteratively applying the rules shown in Fig. 1. We use $\mathcal{DR}^s_\Xi = (\prec_\Xi, \mathcal{DF}^s_\Xi)$ to denote the stable distance relation of Ξ. $\mathcal{DF}^s_\Xi$ maps (e, e') in $\prec_\Xi$ to an interval which corresponds to the lower and upper bounds on the time distance between e and e' in all the behaviours of Ξ.

We represent a partial order by its maximal chains, visually represented by a Hasse diagram. For Ξ_1, $\prec_{\Xi_1}$ is shown in Fig. 2, $\mathcal{DF}^s_{\Xi_1}$ is represented by $\{(h, a, 0, 3), (h, b, 4, 7), (h, c, 4, \infty), (h, d, 4, \infty), (h, e, 4, 7), (h, f, 4, 7), (a, b, 1, 4), (a, c, 1, \infty), (a, d, 1, \infty), (e, b, 0, 3), (e, f, 0, 3), (f, b, 0, 3)\}$. Tuples of the form $(e, e', 0, \infty)$ (indicating default constraints) will not be included in our examples.

Extended distance relations The *extended partial order* of Ξ, $\prec^x_\Xi$: $\Sigma_\Xi \times \Sigma_\Xi$, is the smallest relation that satisfies the following conditions: (i) if $e \prec_\Xi e'$, then $e \prec^x_\Xi e'$; (ii) if there exists $o \in \Sigma_\Xi$ such that $o \prec^x_\Xi e$, $o \prec^x_\Xi e'$ and $M_{oe} < m_{oe'}$ in $\mathcal{DF}^s_\Xi$, or $e \prec^x_\Xi o$, $e' \prec^x_\Xi o$ and $M_{e'o} < m_{eo}$ in $\mathcal{DF}^s_\Xi$, then $e \prec^x_\Xi e'$; (iii) if $e \prec^x_\Xi e'$ and $e' \prec^x_\Xi e''$, then $e \prec^x_\Xi e''$. The *initial* extended distance relation of Ξ is $\mathcal{DR}^x_\Xi = (\prec^x_\Xi, \mathcal{DF}^x_\Xi)$, where $\mathcal{DF}^x_\Xi(e, e') = [0, \infty]$ if (e, e') in $\prec^x_\Xi \setminus \prec_\Xi$, and $\mathcal{DF}^x_\Xi(e, e') = \mathcal{DF}^s_\Xi(e, e')$ if (e, e') in $\prec_\Xi$. The *stable extended distance relation* of Ξ, denoted by $\mathcal{DR}^{xs}_\Xi = (\prec^x_\Xi, \mathcal{DF}^{xs}_\Xi)$, is obtained by stabilising $\mathcal{DR}^x_\Xi$.

In Ξ_1, $M_{ha} = 3 < m_{he} = 4$ in $\mathcal{DF}^s_{\Xi_1}$, so $\prec_{\Xi_1}$ can be extended with (a, e) (see $\prec^x_{\Xi_1}$ in Fig. 2) and $\mathcal{DF}^{xs}_{\Xi_1} = \mathcal{DF}^s_{\Xi_1} \cup \{(a, e, 1, 4), (a, f, 1, 4)\}$.

To avoid visual clutter we will use $\mathcal{DR}_\Xi = (\prec_\Xi, \mathcal{DF}_\Xi)$ for the stable extended distance relation of Ξ. A (sequential) scenario, ξ, can be seen as a trivial DTS, $\mathcal{D}(\xi)$. We use $\mathcal{DR}_\xi = (\prec_\xi, \mathcal{DF}_\xi)$ for its stable distance relation, where $\prec_\xi$ is $<_\xi$.

Realisability of (sequential) scenarios as distributed scenarios Let $\mathcal{S} = \{s_1, \ldots, s_n\}$, such that each $s_i \in \mathcal{S}$ has a different sequence of events, and each event in $\Sigma_\mathcal{S}$ occurs exactly once in each s_i. If there exists a DTS Ξ over $\Sigma_\mathcal{S}$, such that $[\![\Xi]\!] = \bigcup_{1 \leq i \leq n} [\![s_i]\!]$, we say $\mathcal{S}$ is *realisable* by Ξ.

Projections of distance relations Let $\mathcal{DR} = (\prec, \mathcal{DF})$ be a stable distance relation over Σ. A *projection* of $\mathcal{DR}$ is a scenario $\xi_p = (p, \mathcal{C}_p)$, where (i) p is a permutation of events in Σ that agrees with $\prec$, and (ii) $\mathcal{C}_p$ is a set of constraints: for every e and e' such that e appears before e' in p, if $e \not\prec e'$, then the default constraints $0 \leq \tau_{e,e'} \leq \infty$ are added to $\mathcal{C}_p$, otherwise the constraints $l \leq \tau_{e,e'} \leq h$ are added to $\mathcal{C}_p$, where $\mathcal{DF}(e, e') = [l, h]$. When we create $\mathcal{DR}_{\xi_p}$, we must, of

course, stabilise it. In the rest of the text we assume the distance relations of all the projections are stabilised.

The set of all the projections obtained in this way is realisable by any DTS for which $\mathcal{DR}$ is the extended distance relation. Moreover, given the extended distance relation $\mathcal{DR}$ for a DTS Ξ, the set of projections of $\mathcal{DR}$ uniquely determines the semantics of Ξ: every behaviour in $[\![\Xi]\!]$ is allowed by one projection.

3 Timed Scenario Expressions

In this section we introduce the notion of timed scenario expressions (expressions for short). Expressions allow us to construct models of behaviours of complicated systems whose constituent parts are distributed systems. They provide a hierarchical way of specifying systems, where distributed scenarios are used as the building blocks. First, we introduce a few operations for manipulating scenarios.

Definition 1. *Let $\xi = (\mathcal{E}_1, \mathcal{C}_1)$ and $\eta = (\mathcal{E}_2, \mathcal{C}_2)$ be two normalised scenarios over Σ_ξ and Σ_η, respectively.*

- *The* concatenation *of ξ and η, denoted by $\xi \circ \eta$, is defined if $\Sigma_\xi \cap \Sigma_\eta = \emptyset$. If defined, $\xi \circ \eta$ is the scenario $(\mathcal{E}, \mathcal{C}_1 \cup \mathcal{C}_2)$ over $\Sigma_\xi \cup \Sigma_\eta$, where $\mathcal{E}$ is the concatenation of $\mathcal{E}_1$ and $\mathcal{E}_2$. The semantics of $\xi \circ \eta$, denoted by $[\![\xi \circ \eta]\!]$, is $[\![\xi]\!] \circ [\![\eta]\!]$, that is, every behaviour of $\xi \circ \eta$ is a concatenation of some behaviour of ξ and some behaviour of η.*
- *The* choice *between two alternatives, ξ and η, is denoted by $\xi|\eta$. Its semantics is defined by $[\![\xi|\eta]\!] = [\![\xi]\!] \cup [\![\eta]\!]$.*

If ξ and η are two scenarios, then $\xi|\eta$ is consistent if ξ or η is consistent. If $\xi \circ \eta$ is defined, then it is consistent if ξ and η are both consistent.

Definition 2. *Let $\Xi_1 = \mathcal{D}(\xi_1, \ldots, \xi_n)$ and $\Xi_2 = \mathcal{D}(\eta_1, \ldots, \eta_m)$ be two distributed scenarios over Σ_{Ξ_1} and Σ_{Ξ_2}, respectively.*

- *The* concatenation *of Ξ_1 and Ξ_2, denoted by $\Xi_1 \circ \Xi_2$, is defined if $\Sigma_{\Xi_1} \cap \Sigma_{\Xi_2} = \emptyset$. If defined, $\Xi_1 \circ \Xi_2$ is the DTS $\Xi = \mathcal{D}(\gamma_{11}, \ldots, \gamma_{1m}, \ldots, \gamma_{n1}, \ldots, \gamma_{nm})$ over $\Sigma_{\Xi_1} \cup \Sigma_{\Xi_2}$, where $\gamma_{ij} = \xi_i \circ \eta_j$ (for $1 \leq i \leq n$ and $1 \leq j \leq m$), and $[\![\Xi_1 \circ \Xi_2]\!] = [\![\Xi]\!]$.*
- *The* parallel composition *of Ξ_1, and Ξ_2, denoted by $\Xi_1 || \Xi_2$, is the DTS $\Xi = \mathcal{D}(\xi_1, \ldots, \xi_n, \eta_1, \ldots, \eta_m)$ over $\Sigma_{\Xi_1} \cup \Sigma_{\Xi_2}$ and $[\![\Xi_1 || \Xi_2]\!] = [\![\Xi]\!]$.*
- *The* choice *between two alternatives, Ξ_1 and Ξ_2, is denoted by $\Xi_1 | \Xi_2$. Its semantics is defined by $[\![\Xi_1 | \Xi_2]\!] = [\![\Xi_1]\!] \cup [\![\Xi_2]\!]$.*

If Ξ_1 and Ξ_2 are two distributed scenarios, then $\Xi_1 | \Xi_2$ is consistent if Ξ_1 or Ξ_2 is consistent. If $\Xi_1 \circ \Xi_2$ is defined, then it is consistent if Ξ_1 and Ξ_2 are both consistent. It is worth pointing out that, except for the case of concatenation, Ξ_1 and Ξ_2 might share some event names. In that case, the event names in $\Sigma_{\Xi_1} \cap \Sigma_{\Xi_2}$ will be synchronising events in $\Xi_1 || \Xi_2$. If Ξ_1 and Ξ_2 are both consistent, $\Xi_1 || \Xi_2$ might not be well-formed, and, if well-formed, might not be consistent.

For example, $\Xi_1 = \mathcal{D}(\xi_1, \xi_2)$, where $\xi_1 = (abc, \{\tau_{a,b} \geq 2\})$ and $\xi_2 = (eb, \{\tau_{e,b} \leq 3\})$ is a consistent DTS, so is $\Xi_2 = \mathcal{D}(\xi_3)$, where $\xi_3 = (efb, \{\tau_{e,b} \geq 4\})$. But

$\Xi_1||\Xi_2$ is inconsistent. Observe that b is a synchronising event in Ξ_1, while both b and e are the synchronising events in $\Xi_1||\Xi_1$. Because the constraints $\tau_{e,b} \leq 3$ in Ξ_1 and $\tau_{e,b} \geq 4$ in Ξ_2 cannot be satisfied simultaneously, $\Xi_1||\Xi_1$ is inconsistent.

Definition 3. *A* timed scenario expression *(*expression *for short) is one of the following:*
- *a DTS,*
- $(E_1 \circ E_2)$*, where* E_1 *and* E_2 *are expressions,*
- $(E_1||E_2)$*, where* E_1 *and* E_2 *are expressions, or*
- $(E_1|E_2)$*, where* E_1 *and* E_2 *are expressions.*

Recall that a scenario ξ can be seen as the DTS $\mathcal{D}(\xi)$, so it is an expression.

It should be obvious that the three operators, $\circ$, $|$ and $||$, are associative. The first one is not commutative, while the other two are.

The following observation follows directly from the definitions.

Observation 1 *Distributive laws for expressions:*
- $E_1 \circ (E_2||E_3) = (E_1 \circ E_2)||(E_1 \circ E_3)$, $(E_1||E_2) \circ E_3 = (E_1 \circ E_3)||(E_2 \circ E_3)$
- $E_1 \circ (E_2|E_3) = (E_1 \circ E_2)|(E_1 \circ E_3)$, $(E_1|E_2) \circ E_3 = (E_1 \circ E_3)|(E_2 \circ E_3)$
- $E_1||(E_2|E_3) = (E_1||E_2)|(E_1||E_3)$, $(E_1|E_2)||E_3 = (E_1||E_3)|(E_2||E_3)$

Definition 4. *An expression, E, is in* normal form *if E is either a DTS Ξ, or of the form $\Xi_1|\ldots|\Xi_k$, where each Ξ_i $(1 \leq i \leq k, 1 < k)$ is a DTS.*

Using the distributive laws an expression can be converted into normal form. It should be obvious that the normal form of an expression is unique.

Observation 2 *Let E be an expression. The semantics of E, denoted by $[\![E]\!]$, is*
- $[\![\Xi]\!]$*, if E is an expression with normal form Ξ,*
- $[\![\Xi_1]\!] \cup \cdots \cup [\![\Xi_k]\!]$*, if E is an expression with normal form $\Xi_1|\ldots|\Xi_k$ $(1 < k)$.*

Definition 5. *An expression, E, is* consistent*, if $[\![E]\!] \neq \emptyset$.*

If E is a consistent expression, we say $[\![E]\!]$ is the set of behaviours that are *described* or *expressed* by E.

Example 1. Consider a system in which two runners compete with each other in a race managed by a controller [6]. The race begins with the controller issuing a signal `start`, which is received by both the runners simultaneously. Upon completing the run, runner `i` sends a $\texttt{finish}_\texttt{i}$ signal to the controller. So there are two alternative outcomes, depending on which runner finishes first. This can be modeled by the expression $(\texttt{C}_1|\texttt{C}_2)||\texttt{R}_1||\texttt{R}_2$, in which $\texttt{C}_1$ and $\texttt{C}_2$ correspond to the two outcomes, each specified by a DTS: $\texttt{C}_1 = (\texttt{start finish}_1\ \texttt{finish}_2, \emptyset)$, $\texttt{C}_2 = (\texttt{start finish}_2\ \texttt{finish}_1, \emptyset)$, while $\texttt{R}_1$ and $\texttt{R}_2$ capture the behaviours of the runners: $\texttt{R}_1 = (\texttt{start finish}_1, \emptyset)$, $\texttt{R}_2 = (\texttt{start finish}_2, \emptyset)$. Observe that `start` is a synchronising event between all the components, $\texttt{finish}_1$ is a synchronising event between the controller (specified by $(\texttt{C}_1|\texttt{C}_2)$) and $\texttt{R}_1$, while $\texttt{finish}_2$ is a synchronising event between the controller and $\texttt{R}_2$.

In the rest of the paper our examples will not feature the operator $\circ$, as it plays no role in the problem of realisation.

4 Realisation of Sequential Scenarios as Expressions

In our earlier work [8] we addressed the question of whether a set $\mathcal{S}$ of normalised (sequential) scenarios, whose sequences of events are different permutations of the same set, can be realised as a single DTS that is semantically equivalent to $\mathcal{S}$, i.e., describes the same set of behaviours as S. This can be done only if there is a distance relation whose set of projections is the same as $\mathcal{S}$. We showed how to obtain such a distance relation and a corresponding DTS [8].

In this section we tackle the problem of realisability of scenarios *as expressions*: given a set, $\mathcal{S}$, of scenarios, we want to find an expression that realises $\mathcal{S}$. Despite similarities, there is a fundamental difference between this problem and the one previously solved [8]: the members of $\mathcal{S}$ could be projections that are obtained from extended distance relations of different distributed scenarios, in which case $\mathcal{S}$ cannot be realised by a single DTS, but by a choice expression that involves more than one DTS. The challenge is to divide $\mathcal{S}$ in such a way that each partition could be realised by a DTS, while trying to avoid the trivial solution in which every member of S is a separate alternative. More precisely:

> Given a finite set $\mathcal{S} = \{s_1, \ldots, s_n\}$ of scenarios of *possibly different lengths* over Σ, such that (i) $eseq(s_i) \neq eseq(s_j)$ (for $1 \leq i, j \leq n$ and $i \neq j$)[6], and (ii) each event of s_i ($1 \leq i \leq n$) has a unique occurrence in s_i, we want to obtain an expression E over Σ, such that $[\![E]\!] = [\![\mathcal{S}]\!] = \bigcup_{1 \leq i \leq n} [\![s_i]\!]$. In general, $\mathcal{S}$ can be realised by more than one expression. Our goal is to find an expression with a reasonably small number of alternatives, while decreasing the number of trivial alternatives.

We say $\mathcal{S}$ is *realisable by* E, or E is a *realisation* of $\mathcal{S}$, if $\mathcal{S}$ is realised by E.

Example 2. Given the set, $\mathcal{S} = \{s_1, s_2, s_3, s_4\}$, of scenarios, where $s_1 = (\texttt{start}\ \texttt{finish}_1\ \texttt{finish}_2\ \texttt{flag}_1, \emptyset)$, $s_2 = (\texttt{start}\ \texttt{finish}_1\ \texttt{flag}_1\ \texttt{finish}_2, \emptyset)$, $s_3 = (\texttt{start}\ \texttt{finish}_2\ \texttt{finish}_1\ \texttt{flag}_2, \emptyset)$, $s_4 = (\texttt{start}\ \texttt{finish}_2\ \texttt{flag}_2\ \texttt{finish}_1, \emptyset)$, there is no DTS that realises $\mathcal{S}$. But $\mathcal{S}$ can be divided into two sets $\mathcal{S}_1 = \{s_1, s_2\}$ and $\mathcal{S}_2 = \{s_3, s_4\}$ based on the sets of events of its members. The set $\mathcal{S}_1$ can be realised by DTS $E = \mathcal{D}(\xi_1, \xi_2) = \xi_1 || \xi_2$, where $\xi_1 = (\texttt{start}\ \texttt{finish}_1\ \texttt{flag}_1, \emptyset)$ and $\xi_2 = (\texttt{start}\ \texttt{finish}_1\ \texttt{finish}_2, \emptyset)$. Similarly, $\mathcal{S}_2$ is realised by DTS $E' = \xi_3 || \xi_4$, where $\xi_3 = (\texttt{start}\ \texttt{finish}_2\ \texttt{flag}_2, \emptyset)$ and $\xi_4 = (\texttt{start}\ \texttt{finish}_2\ \texttt{finish}_1, \emptyset)$. So $\mathcal{S}$ is realised by the expression $E|E' = (\xi_1 || \xi_2)|(\xi_3 || \xi_4)$.

Definition 6. *Let $\mathcal{S} = \{s_1, \ldots, s_n\}$ be a finite set of normalised scenarios and $\{\mathcal{DR}_{s_1}, \ldots, \mathcal{DR}_{s_n}\}$ be the set of corresponding stable distance relations, where $\mathcal{DR}_{s_i} = (\prec_{s_i}, \mathcal{DF}_{s_i})$ ($1 \leq i \leq n$).* The distance relation of *$\mathcal{S}$, $\mathcal{DR}_{\mathcal{S}}$, is the pair $(\prec_{\mathcal{S}}, \mathcal{DF}_{\mathcal{S}})$, where $\prec_{\mathcal{S}} = \bigcap_{1 \leq i \leq n} \prec_{s_i}$ and, for every (e, e') in $\prec_{\mathcal{S}}$, $\mathcal{DF}_{\mathcal{S}}(e, e')$ is the smallest interval that includes each $\mathcal{DF}_{s_i}(e, e')$.*

The first step in solving the problem posed at the beginning of this section is to divide $\mathcal{S}$ in such a way that all the members of each partition are scenarios of equal lengths over the same set of events. Then we solve the problem for

[6] In Sec. 4.4 we show how to relax the assumption.

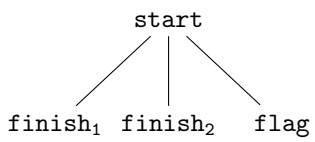

Fig. 3: Maximal chains (Example 3)

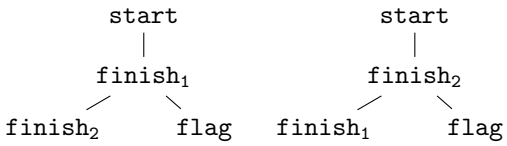

Fig. 4: Maximal chains for $\prec_{\mathcal{S}_1}$, $\prec_{\mathcal{S}_2}$(Example 3)

each partition, independently. Our general strategy is the following: whenever a partition is not realisable by a single DTS, we divide it repeatedly, so that at the end each partition will be realised by a DTS and will correspond to one alternative in the overall choice expression. This process is guided by the results of our analysis based on $\prec_{\mathcal{S}}$ and $\mathcal{DF}_{\mathcal{S}}$, described in sections 4.1 and 4.2. Additionally, some divisions are simply necessary, as described below.

Gaps Let $S_{(e,e')} = \{s_i \in \mathcal{S} | e \prec_{s_i} e'\}$. Let $s_1, \ldots, s_m$ be a sequence of members of $S_{(e,e')}$ such that, for every $1 \leq i \leq m$, $[l_i, h_i] = \mathcal{DF}_{s_i}(e, e')$ and $l_i \leq l_{i+1}$. Let $H_r = \max\{h_j | 1 \leq j \leq r\}$. If there is a k, $1 \leq k < m$, such that $H_k < l_{k+1}$, $\mathcal{DF}_S(e, e')$ contains a gap. Then $\mathcal{S}$ is said to be *with gaps*. An $\mathcal{S}$ with gaps cannot be realised by a single DTS: this is because the interval between e and e' in $\mathcal{DF}_{\mathcal{S}}$ would be $[l_1, H_m]$, which would allow also behaviours in which the time distance between e and e' is between H_k and l_{k+1}. Such behavious are allowed by none of the members of $\mathcal{S}$. So $\mathcal{S}$ must be divided into partitions with no gaps.

Whenever a partition is divided into finer partitions (see Sec. 4.1 and Sec. 4.2), we must check for the existence of gaps in any newly formed partition.

In the rest of the paper we assume that all scenarios in $\mathcal{S}$ are over the same set, $\Sigma_{\mathcal{S}}$, of events, and that each $s \in \mathcal{S}$ contains all the events from $\Sigma_{\mathcal{S}}$. Moreover, we assume that any divisions necessitated by gaps have already been carried out, so that $\mathcal{S}$ is without gaps. We describe our method in two steps: first, we tackle the problem *without accounting for constraints*. Because constraints will not play a role, in our examples we will discuss scenarios with only default constraints.

4.1 Partitioning a Set of Sequential Scenarios Based on Ordering

Let $\mathcal{S} = \{s_1, \ldots s_n\}$ be a set of normalised scenarios over $\Sigma_{\mathcal{S}}$, such that, for every $1 \leq i \leq n$, $s_i = (\mathcal{E}_i, \emptyset)$, where $\mathcal{E}_i$ contains all the events from $\Sigma_{\mathcal{S}}$. Moreover, let $\mathcal{S}$ be without gaps. We are interested in obtaining an expression that realises $\mathcal{S}$.

Example 3. Let $\mathcal{S} = \{s_1, s_2, s_3, s_4\}$, where $s_1 = (\texttt{start finish}_1 \texttt{ finish}_2 \texttt{ flag}, \emptyset)$, $s_2 = (\texttt{start finish}_1 \texttt{ flag finish}_2, \emptyset)$, $s_3 = (\texttt{start finish}_2 \texttt{ finish}_1 \texttt{ flag}, \emptyset)$ and $s_4 = (\texttt{start finish}_2 \texttt{ flag finish}_1, \emptyset)$. The set $\mathcal{S}$ cannot be realised by a single DTS, as will be explained below. Unlike in Example 2, $\mathcal{S}$ cannot be divided according to sets of events: each member contains the same set of events. So we need another criterion for partitioning $\mathcal{S}$.

Definition 7. *Two (sequential) scenarios, s and p,* match *if $eseq(s) = eseq(p)$.*

Definition 8. *Let $\mathcal{S}$ be a finite set of scenarios and $\mathcal{DR}_{\mathcal{S}}$ be its distance relation. The set of projections of $\mathcal{DR}_{\mathcal{S}}$ (see Sec. 2.2) is denoted by $\mathcal{P}_{\mathcal{S}}$.*

Definition 9. *Let $\mathcal{S}$ be a finite set of scenarios. We say $\mathcal{S}$ is* order-closed *iff every member of $\mathcal{P}_{\mathcal{S}}$ matches one scenario in $\mathcal{S}$ and vice versa.*

Intuitively, if $\mathcal{S}$ is order-closed, then extending it with a scenario that has a new sequence of events will change $\prec_{\mathcal{S}}$.

Observation 3 *Let $\mathcal{S}$ be a finite set of scenarios. If $|\mathcal{S}| = |\mathcal{P}_{\mathcal{S}}|$, then $\mathcal{S}$ is order-closed.*

As mentioned at the beginning of Sec. 4, $\mathcal{S}$ can be realised as a single DTS only if $\mathcal{S} = \mathcal{P}_{\mathcal{S}}$. This leads to the following observation:

Observation 4 *Let $\mathcal{S}$ be a finite set of scenarios. If $\mathcal{S}$ is not order-closed, then it cannot be realised by a single DTS.*

In Example 3, $\prec_{\mathcal{S}}$ is represented by the maximal chains $\texttt{start}-\texttt{finish}_1$, $\texttt{start}-\texttt{finish}_2$ and $\texttt{start} - \texttt{flag}$ (see Fig. 3). The distance function of $\mathcal{S}$, $\mathcal{DF}_{\mathcal{S}}$, has only default constraints. The set of projections of $\mathcal{DR}_{\mathcal{S}} = (\prec_{\mathcal{S}}, \mathcal{DF}_{\mathcal{S}})$, includes four projections that match the scenarios in $\mathcal{S}$. Additionally, $\mathcal{P}_{\mathcal{S}}$ includes two projections whose sequences of events are $\texttt{start flag finish}_1\ \texttt{finish}_2$ and $\texttt{start flag finish}_2\ \texttt{finish}_1$. So, by Definition 9, $\mathcal{S}$ is not order-closed.

In general, $\prec_{\mathcal{S}}$ captures the ordering relations that are identical between all the members of $\mathcal{S}$. If $\mathcal{S}$ is not order-closed, there are certain subtle similarities between some of the members, similarities that cannot be explained by $\prec_{\mathcal{S}}$. For instance, in Example 3, $\prec_{\mathcal{S}}$ does not reflect the fact that `flag` does not occur immediately after `start` in any member of $\mathcal{S}$: `flag` is preceded by $\texttt{finish}_1$, $\texttt{finish}_2$, or both in all the members of $\mathcal{S}$.

Intuitively, if $\mathcal{S}$ is not order-closed, $\prec_{\mathcal{S}}$ is *too permissive*. So $\prec_{\mathcal{S}}$ would have to be made stricter, somehow, to reflect *all* the ordering relations that exist between the members of $\mathcal{S}$. But this cannot be achieved without partitioning $\mathcal{S}$. Our goal is to divide $\mathcal{S}$ in such a way that each partition is order-closed. To achieve this we investigate the relations between events of $\Sigma_{\mathcal{S}}$ that are not related by $\prec_{\mathcal{S}}$.

Definition 10. *Let $\mathcal{S}$ be a finite set of scenarios over $\Sigma_{\mathcal{S}}$, such that $\mathcal{S}$ is not order-closed. Let $e, e' \in \Sigma_{\mathcal{S}}$, such that $(e, e') \notin \prec_{\mathcal{S}}$. The* order-based division *(*OBD *for short) of $\mathcal{S}$ with respect to (e, e'), denoted by $obd_{\mathcal{S}}(e, e')$, is the unordered pair $(\mathcal{S}_1, \mathcal{S}_2)$, where $\mathcal{S}_1 = \{s_1 \in \mathcal{S} \mid e \prec_{s_1} e'\}$ and $\mathcal{S}_2 = \{s_2 \in \mathcal{S} \mid e' \prec_{s_2} e\}$.*

Observe that all scenarios in $\mathcal{S}$ contain e and e', but not all in the same order. Clearly, an OBD $(\mathcal{S}_1, \mathcal{S}_2)$ is a partitioning, i.e., $\mathcal{S}_1 \neq \emptyset$, $\mathcal{S}_2 \neq \emptyset$ and $\mathcal{S}_1 \cup \mathcal{S}_2 = \mathcal{S}$.

Observation 5 *Let $\mathcal{S}$ be a finite set of scenarios, such that $\mathcal{S}$ is not order-closed and $|\mathcal{S}| > 1$. Then there is at least one OBD.*

In Example 3 $\texttt{finish}_1 \not\prec_{\mathcal{S}} \texttt{finish}_2$ and $obd_{\mathcal{S}}(\texttt{finish}_1, \texttt{finish}_2) = (\mathcal{S}_1, \mathcal{S}_2)$, where $\mathcal{S}_1 = \{s_1, s_2\}$ and $\mathcal{S}_2 = \{s_3, s_4\}$. Notice that $\texttt{finish}_1$ occurs before `flag` in all members of $\mathcal{S}_1$, while in all members of $\mathcal{S}_2$ $\texttt{finish}_2$ occurs before `flag`. So $\prec_{\mathcal{S}_1} = \prec_{\mathcal{S}} \cup \{(\texttt{finish}_1, \texttt{finish}_2), (\texttt{finish}_1, \texttt{flag})\}$ and

$\prec_{\mathcal{S}_2} = \prec_{\mathcal{S}} \cup \{(\texttt{finish}_2, \texttt{finish}_1), (\texttt{finish}_2, \texttt{flag})\}$.

Fig. 4 shows the maximal chains representing $\prec_{\mathcal{S}_1}$ and $\prec_{\mathcal{S}_2}$. Observe that both $\mathcal{S}_1$ and $\mathcal{S}_2$ are order-closed. Indeed, $\mathcal{S}_1$ can be realised by the DTS $\xi_1||\xi_2$, where $\xi_1 = (\texttt{start finish}_1\ \texttt{flag}, \emptyset)$ and $\xi_2 = (\texttt{start finish}_1\ \texttt{finish}_2, \emptyset)$. Similarly, $\mathcal{S}_2$ is realised by the DTS $\xi_3||\xi_4$, where $\xi_3 = (\texttt{start finish}_2\ \texttt{flag}, \emptyset)$ and $\xi_4 = (\texttt{start finish}_2\ \texttt{finish}_1, \emptyset)$. So $\mathcal{S}$ is realised by $(\xi_1||\xi_2)|(\xi_3||\xi_4)$.

In this example there are two other ways to divide $\mathcal{S}$:
$obd_{\mathcal{S}}(\texttt{finish}_1, \texttt{flag}) = (\{s_1, s_2, s_4\}, \{s_3\})$, from which we obtain the expression $(\xi_5||\xi_6)|s_3$, where $\xi_5 = (\texttt{start finish}_1\ \texttt{flag}, \emptyset)$ and $\xi_6 = (\texttt{start finish}_2, \emptyset)$.
$obd_{\mathcal{S}}(\texttt{finish}_2, \texttt{flag}) = (\{s_1, s_3, s_4\}, \{s_2\})$, which results in $(\xi_7||\xi_8)|s_2$, where $\xi_7 = (\texttt{start finish}_2\ \texttt{flag}, \emptyset)$ and $\xi_8 = (\texttt{start finish}_1, \emptyset)$. But such solutions do not align with our goal: our algorithm (see Sec. 4.3) will produce the first solution.

Definition 11. *The* o-ranking *of* $(\mathcal{S}_1, \mathcal{S}_2)$*, a division of* $\mathcal{S}$*, is* $o_1 * \frac{|\mathcal{S}_1|}{|\mathcal{S}|} + o_2 * \frac{|\mathcal{S}_2|}{|\mathcal{S}|}$*, where* o_i *(*$i \in \{1, 2\}$*) is 0 if* $\mathcal{S}_i$ *is order-closed, and 1 otherwise.*

This ranking will be used to choose between several possible divisions: lower o-ranking should give more satisfactory results. The intuition is that an order-closed partition is likely to require less additional partitioning. Moreover, the larger the order-closed partition, the better.

Observe that in Example 3 all the OBDs have equal o-ranking. So in order to choose the best division we must find another criterion that looks more deeply into the partitions of OBDs.

The following definition is an adaptation of *Kendall tau distance*[17].

Definition 12. *Let* s_1 *and* s_2 *be two normalised scenarios of equal length over the same set of events. The* K-distance *of* s_1 *and* s_2*, denoted by* $d_K(s_1, s_2)$*, is* $|\{(e, e') \mid e \prec_{s_1} e' \wedge e' \prec_{s_2} e\}|$.

The larger the K-distance, the more dissimilar are the sequences of events.

Definition 13. *Let* $\mathcal{S}$ *be a finite set of scenarios. The* K-ranking *of* $\mathcal{S}$*, denoted by* $r_K(\mathcal{S})$*, is the average of all pairwise K-distances between the members of* $\mathcal{S}$*.*

Since $d_K(s_1, s_2) = d_K(s_2, s_1)$, the average is taken over the number of unordered pairs in $\mathcal{S}$, i.e., $\frac{|\mathcal{S}| * (|\mathcal{S}| - 1)}{2}$.

Definition 14. *Let* $\mathcal{S}$ *be a finite set of scenarios and* $D = (\mathcal{S}_1, \mathcal{S}_2)$ *be an OBD of* $\mathcal{S}$*. The* K-ranking *of* D*, denoted by* $r_K(D)$*, is given by* $r_K(\mathcal{S}_1) * \frac{|\mathcal{S}_1|}{|\mathcal{S}|} + r_K(\mathcal{S}_2) * \frac{|\mathcal{S}_2|}{|\mathcal{S}|}$.

It turns out that the K-rankings of divisions provide us with the crucial information that we need in order to choose the best division when there are multiple divisions that all agree with each other on their o-ranking.

Example 4. Let $\mathcal{S} = \{s_1, s_2, s_3, s_4, s_5, s_6\}$, where $s_1 = (adbc, \emptyset)$, $s_2 = (dabc, \emptyset)$, $s_3 = (acdb, \emptyset)$, $s_4 = (acbd, \emptyset)$, $s_5 = (abcd, \emptyset)$ and $s_6 = (abdc, \emptyset)$. The set $\mathcal{S}$ is not order-closed, so it cannot be realised by a single DTS. The relation $\prec_{\mathcal{S}}$ is represented by the maximal chains $a - b$, $a - c$ and $a - d$. There are four pairs of events that are unrelated by $\prec_{\mathcal{S}}$: (a, d), (b, c), (b, d) and (c, d). The divisions

pair	OBD	order-closed	o-rank	K-rank	expression
(a,d)	$\{s_1, s_3, s_4, s_5, s_6\}$ $\{s_2\}$	No Yes	0.83	1.5	$(\xi_1 \| \xi_2) \mid (\xi_3 \| \xi_4) \mid s_2$ $\xi_1 = (abc, \emptyset), \xi_2 = (ad, \emptyset)$ $\xi_3 = (acb, \emptyset), \xi_4 = (acd, \emptyset)$
(b,c)	$\{s_1, s_2, s_5, s_6\}$ $\{s_3, s_4\}$	Yes Yes	0	1.44	$(\xi_1 \| \xi_2) \mid (\xi_3 \| \xi_4)$ $\xi_1 = (abc, \emptyset), \xi_2 = (d, \emptyset)$ $\xi_3 = (acb, \emptyset), \xi_4 = (acd, \emptyset)$
(b,d)	$\{s_1, s_2, s_3\}$ $\{s_4, s_5, s_6\}$	No Yes	0.5	1.67	$(\xi_1 \| \xi_2) \mid s_3 \mid (\xi_3 \| \xi_4)$ $\xi_1 = (abc, \emptyset), \xi_2 = (dbc, \emptyset)$ $\xi_3 = (abd, \emptyset), \xi_4 = (ac, \emptyset)$
(c,d)	$\{s_1, s_2, s_6\}$ $\{s_3, s_4, s_5\}$	Yes Yes	0	1.34	$(\xi_1 \| \xi_2) \mid (\xi_3 \| \xi_4)$ $\xi_1 = (abc, \emptyset), \xi_2 = (dc, \emptyset)$ $\xi_3 = (ab, \emptyset), \xi_4 = (acd, \emptyset)$

Table 1: Divisions, their o-rankings, K-rankings and resulting expressions (Example 4)

with respect to these pairs are shown in the second column of Table 1, the third column shows whether each partition of a division is order-closed or not, while the fourth column shows the o-rankings. Observe that $obd_{\mathcal{S}}(b,c)$ and $obd_{\mathcal{S}}(c,d)$ have equal and lowest o-rankings. So we compute their K-rankings.
For $obd_{\mathcal{S}}(c,d)$, $r_K(\{s_1, s_2, s_6\}) = \frac{d_K(s_1,s_2)+d_K(s_1,s_6)+d_K(s_2,s_6)}{3} \approx 1.33$. Similarly, $r_K(\{s_3, s_4, s_5\}) \approx 1.33$. So $r_K(obd_{\mathcal{S}}(c,d)) = 1.33 * \frac{3}{6} + 1.33 * \frac{3}{6} \approx 1.34$. Similarly, $r_K(obd_{\mathcal{S}}(b,c)) \approx 1.44$. So we choose $obd_{\mathcal{S}}(c,d)$ that has a lower K-ranking. The resulting expression is shown in the last row, last column of the table: $\{s_1, s_2, s_6\}$ is realised by $\xi_1 \| \xi_2$, while $\{s_3, s_4, s_5\}$ is realised by $\xi_3 \| \xi_4$.

The K-rankings of the other two divisions are shown in the fifth column, while the last column shows the corresponding expressions. Notice that those partitions of $obd_{\mathcal{S}}(a,d)$ and $obd_{\mathcal{S}}(b,d)$ that are not order-closed must be further partitioned. Observe that $obd_{\mathcal{S}}(b,c)$ would have also resulted in an expression with two alternatives. In general, divisions with lowest K-rankings produce more satisfactory results, unless their o-rankings differ.

In our algorithm (see Sec. 4.3) we choose a division with the lowest o-ranking. We break a tie by choosing a division with the lowest K-ranking. In order to further break a tie we introduce a new ranking, called l-ranking, that takes into account whether the distance functions of the partitions of a division include gaps or not, and how balanced the partitions are with respect to their sizes. For a division D, its l-ranking is computed by $100 * g + b$, where g is the number of partitions of D that include gaps, and b is the difference between the sizes of the two partitions of D. The aim is to give higher priority to divisions whose partitions are without gaps. Parameter b can be used to discard trivial partitions, in favour of non-trivial ones. (We assume $|S| < 100$.)

4.2 Partitioning a Set of Sequential Scenarios Involving Constraints

In this section we fully tackle the problem of realisability of sets of scenarios as expressions, that is we account also for constraints. We begin with an example.

Example 5. Let $\mathcal{S} = \{s_1, s_2, s_3\}$ over $\Sigma = \{a, b, c, d\}$, where $s_1 = (cdab, \{\tau_{c,b} \leq 3\})$, $s_2 = (cadb, \{\tau_{c,d} \leq 1, \tau_{c,b} \leq 3\})$ and $s_3 = (cabd, \{\tau_{c,d} \leq 1\})$. The set $\mathcal{S}$ is order-closed, but, as will be explained below, it is not realisable by a single DTS. Therefore, $\mathcal{S}$ must be partitioned, somehow. The pairs of events not related by $\prec_{\mathcal{S}}$ are (a, d) and (b, d), based on which two different OBDs, $(\{s_2, s_3\}, \{s_1\})$ and $(\{s_1, s_2\}, \{s_3\})$, could be formed. The divisions have not only equal o-rankings, but also equal K-rankings and l-rankings. So a division must be chosen randomly. Choosing the second division results in the trivial expression $s_1|s_2|s_3$. But, in fact, a satisfactory result can be obtained from the first division.

This example shows that in the presence of non-default constraints, when $\mathcal{S}$ is order-closed, but not realisable by a single DTS, reasoning based on orderings alone does not provide us with enough information to properly partition $\mathcal{S}$. Next, we investigate how the constraints can be used to effectively partition $\mathcal{S}$.

Definition 15. *Let $\mathcal{S}$ be an order-closed set of scenarios. Let $sc \in \mathcal{S}$ and $p \in \mathcal{P}_{\mathcal{S}}$, such that sc matches p. We say sc is* compatible with $\mathcal{DR}_{\mathcal{S}}$ *if $\mathcal{DF}_{sc} = \mathcal{DF}_p$.*

Notice that if all the members of $\mathcal{S}$ have only default constraints, then they are all compatible with $\mathcal{DR}_{\mathcal{S}}$.

In Example 5 $\prec_{s_1}$ is represented by $c - d - a - b$, while $\prec_{s_2}$ and $\prec_{s_3}$ are represented by $c-a-d-b$ and $c-a-b-d$, respectively. The distance functions of members of $\mathcal{S}$ are shown below:

$\mathcal{DF}_{s_1} = \{(a, b, 0, 3), (c, a, 0, 3), (c, b, 0, 3), (c, d, 0, 3), (d, a, 0, 3), (d, b, 0, 3)\}$,
$\mathcal{DF}_{s_2} = \{(a, b, 0, 3), (a, d, 0, 1), (c, a, 0, 1), (c, b, 0, 3), (c, d, 0, 1), (d, b, 0, 3)\}$,
$\mathcal{DF}_{s_3} = \{(a, b, 0, 1), (a, d, 0, 1), (b, d, 0, 1), (c, a, 0, 1), (c, b, 0, 1), (c, d, 0, 1)\}$.

The stable distance relation of $\mathcal{S}$ (see Def. 6) is $\mathcal{DR}_{\mathcal{S}} = (\prec_{\mathcal{S}}, \mathcal{DF}_{\mathcal{S}})$, where $\prec_{\mathcal{S}} = \prec_{s_1} \cap \prec_{s_2} \cap \prec_{s_3}$ is represented by $c - a - b$ and $c - d$, and $\mathcal{DF}_{\mathcal{S}}$ is represented by $\{(a, b, 0, 3), (c, a, 0, 3), (c, b, 0, 3), (c, d, 0, 3)\}$. Observe that, for example, $\mathcal{DF}_{\mathcal{S}}(a, b) = [0, 3]$ is the smallest interval that includes $\mathcal{DF}_{s_1}(a, b)$, $\mathcal{DF}_{s_2}(a, b)$ and $\mathcal{DF}_{s_3}(a, b)$. The set of projections of $\mathcal{DR}_{\mathcal{S}}$ (see Sec. 2.2) is $\mathcal{P}_{\mathcal{S}} = \{p_1, p_2, p_3\}$, where p_1, p_2 and p_3 match s_1, s_2 and s_3, respectively. The distance functions of the projections in $\mathcal{P}_{\mathcal{S}}$ are shown below:

$\mathcal{DF}_{p_1} = \{(a, b, 0, 3), (c, a, 0, 3), (c, b, 0, 3), (c, d, 0, 3), (d, a, 0, 3), (d, b, 0, 3)\}$,
$\mathcal{DF}_{p_2} = \{(a, b, 0, 3), (a, d, 0, 3), (c, a, 0, 3), (c, b, 0, 3), (c, d, 0, 3), (d, b, 0, 3)\}$,
$\mathcal{DF}_{p_3} = \{(a, b, 0, 3), (a, d, 0, 3), (b, d, 0, 3), (c, a, 0, 3), (c, b, 0, 3), (c, d, 0, 3)\}$.

Notice that $\mathcal{DF}_{s_1} = \mathcal{DF}_{p_1}$, so s_1 is compatible with $\mathcal{DR}_{\mathcal{S}}$, but s_2 and s_3 are not. This observation can be immediately used to divide $\mathcal{S}$ into $\mathcal{S}_1 = \{s_1\}$ and $\mathcal{S}_2 = \{s_2, s_3\}$. The set $\mathcal{S}_2$ is order-closed, its distance relation is $\mathcal{DR}_{\mathcal{S}_2} = (\prec_{\mathcal{S}_2}, \mathcal{DF}_{\mathcal{S}_2})$, where $\prec_{\mathcal{S}_2}$ is represented by the maximal chains $c-a-b$ and $c-a-d$, and $\mathcal{DF}_{\mathcal{S}_2}$ is represented by $\{(a, b, 0, 3), (a, d, 0, 1), (c, a, 0, 1), (c, b, 0, 3), (c, d, 0, 1)\}$. The set of projections of $\mathcal{DR}_{\mathcal{S}_2}$, $\mathcal{P}_{\mathcal{S}_2}$, includes two projections, p'_2 and p'_3, that are identical to s_2 and s_3, respectively, so $\mathcal{DF}_{s_2} = \mathcal{DF}_{p'_2}$ and $\mathcal{DF}_{s_3} = \mathcal{DF}_{p'_3}$: s_2 and s_3 are compatible with $\mathcal{DR}_{\mathcal{S}_2}$. The set $\mathcal{S}_2$ is realised by DTS $\xi_1||\xi_2$, where $\xi_1 = (cab, \{\tau_{c,a} \leq 1, \tau_{c,b} \leq 3\})$ and $\xi_2 = (cad, \{\tau_{c,d} \leq 1\})$. So $\mathcal{S}$ is realised by $s_1|(\xi_1||\xi_2)$ (compare this with the trivial solution shown in Example 5).

In Example 5 we partitioned $\mathcal{S}$ into scenarios that are compatible with $\mathcal{DR}_{\mathcal{S}}$, and those that are not. But sometimes none of the members of $\mathcal{S}$ will be compatible with $\mathcal{DR}_{\mathcal{S}}$, so we need another criterion for partitioning.

Definition 16. *Let $\mathcal{S}$ be an order-closed set of scenarios and $sc \in \mathcal{S}$. The* constraint-based disagreement *between sc and $\mathcal{DR}_{\mathcal{S}}$ is $dsg_{\mathcal{S}}(sc) = \{(e, e') \mid e \prec_{sc} e' \wedge \mathcal{DF}_{sc}(e, e') \neq \mathcal{DF}_p(e, e')$ where p is the projection of $\mathcal{DR}_{\mathcal{S}}$ that matches $sc\}$.*

Definition 17. *Let $\mathcal{S}$ be an order-closed set of scenarios over $\Sigma_{\mathcal{S}}$, such that none of its members are compatible with $\mathcal{DR}_{\mathcal{S}}$ and $|\mathcal{S}| > 1$. Let $e, e' \in \Sigma_{\mathcal{S}}$. The* constraint-based division *(*CBD *for short)* of $\mathcal{S}$ with respect to (e, e'), *denoted by $cbd_{\mathcal{S}}(e, e')$, is the unordered pair $(\mathcal{S}_1, \mathcal{S}_2)$, such that*
(1) $\mathcal{S}_1 = \{sc \in \mathcal{S} \mid (e, e') \in dsg_{\mathcal{S}}(sc)\}$ and $\mathcal{S}_2 = \mathcal{S} \setminus \mathcal{S}_1$, or
(2) $(e, e') \notin dsg_{\mathcal{S}}(sc)$ for any $sc \in \mathcal{S}$ and there exists some $sc' \in \mathcal{S}$, where $e \prec_{sc'} e' \wedge \mathcal{DF}_{sc'}(e, e') \neq [0, \infty]$, then $\mathcal{S}_1 = \{s \in \mathcal{S} \mid e \prec_s e'\}$ and $\mathcal{S}_2 = \mathcal{S} \setminus \mathcal{S}_1$.

Intuitively, if $cbd_{\mathcal{S}}(e, e') = (\mathcal{S}_1, \mathcal{S}_2)$ is formed by case (1) of Def. 17, then all the members of $\mathcal{S}_1$ disagree with their respective matching projections on the interval for (e, e'), i.e., on $\mathcal{DF}(e, e')$. In this case, if $s_2 \in \mathcal{S}_2$, then either $\mathcal{DF}_{s_2}(e, e') = \mathcal{DF}_{p_2}(e, e')$, where p_2 is the projection that matches s_2, or $e \not\prec_{s_2} e'$. Notice that $\mathcal{S}_1 \neq \emptyset$, but $\mathcal{S}_2$ can be empty. If $\mathcal{S}_2 = \emptyset$, then $(\mathcal{S}_1, \mathcal{S}_2)$ is *not* a CBD.

If $cbd_{\mathcal{S}}(e, e') = (\mathcal{S}_1, \mathcal{S}_2)$ is formed by case (2), then all the members of $\mathcal{S}_1$ agree with their respective matching projections on the interval for (e, e'), however, at least one of them has a non-default interval on (e, e').

Example 6. Let $\mathcal{S} = \{s_1, s_2\}$, where $s_1 = (abc, \{1 \leq \tau_{a,b} \leq 2\})$ and $s_2 = (acb, \{2 \leq \tau_{a,b} \leq 3\})$. The partial order in $\mathcal{S}$, $\prec_{\mathcal{S}} = \prec_{s_1} \cap \prec_{s_2}$, is represented by $a - b$ and $a - c$. The distance functions of the members of $\mathcal{S}$ are $\mathcal{DF}_{s_1} = \{(a, b, 1, 2), (a, c, 1, \infty)\}$ and $\mathcal{DF}_{s_2} = \{(a, b, 2, 3), (a, c, 0, 3), (c, b, 0, 3)\}$.

Observe that the smallest interval that includes $\mathcal{DF}_{s_1}(a, b) = [1, 2]$ and $\mathcal{DF}_{s_2}(a, b) = [2, 3]$ is $[1, 3]$, so $\mathcal{DF}_{\mathcal{S}}(a, b) = [1, 3]$. Similarly, $\mathcal{DF}_{\mathcal{S}}(a, c) = [0, \infty]$ is the smallest interval that includes $\mathcal{DF}_{s_1}(a, c) = [1, \infty]$ and $\mathcal{DF}_{s_2}(a, c) = [0, 3]$. Therefore, $\mathcal{DF}_{\mathcal{S}}$ is represented by $\{(a, b, 1, 3)\}$ (recall that we do not show the default intervals).

The projections of $\mathcal{DR}_{\mathcal{S}}$ yield $\mathcal{DF}_{p_1} = \{(a, b, 1, 3), (a, c, 1, \infty)\}$ and $\mathcal{DF}_{p_2} = \{(a, b, 1, 3), (a, c, 0, 3), (c, b, 0, 3)\}$, where s_1 matches p_1 and s_2 matches p_2. None of the members of $\mathcal{S}$ are compatible with $\mathcal{DR}_{\mathcal{S}}$: $dsg_{\mathcal{S}}(s_1) = dsg_{\mathcal{S}}(s_2) = \{(a, b)\}$. By case (1) of Def. 17, $cbd_{\mathcal{S}}(a, b) = (\{s_1, s_2\}, \emptyset)$. However, $\mathcal{DF}_{s_1}(b, c) = [0, \infty]$ and $\mathcal{DF}_{s_2}(c, b) = [0, 3] \neq [0, \infty]$, so, by case (2) of Def. 17, $cbd_{\mathcal{S}}(b, c) = (\{s_1\}, \{s_2\})$.

Theorem 1. *Let $\mathcal{S}$ be an order-closed set of scenarios, such that none of its members is compatible with $\mathcal{DR}_{\mathcal{S}}$ and $|\mathcal{S}| > 1$. Then there is at least one pair of events e and e' such that $cbd_{\mathcal{S}}(e, e') = (\mathcal{S}_1, \mathcal{S}_2)$, where $\mathcal{S}_1 \neq \emptyset$ and $\mathcal{S}_2 \neq \emptyset$.*

In general, $\mathcal{S}$ might have more than one CBD, so we need a criterion for comparing them.

Definition 18. *Let $\mathcal{S}$ be an order-closed set of scenarios and $s_i, s_j \in \mathcal{S}$ $(i \neq j)$. The* c-distance *of s_i and s_j, denoted by $d_c(s_i, s_j)$, is $|\{cbd_{\mathcal{S}}(e, e') = (\mathcal{S}_1, \mathcal{S}_2) \mid \mathcal{S}_1 \neq \emptyset \wedge \mathcal{S}_2 \neq \emptyset \wedge ((s_i \in \mathcal{S}_1 \wedge s_j \in \mathcal{S}_2) \vee (s_i \in \mathcal{S}_2 \wedge s_j \in \mathcal{S}_1))\}|$.*

Intuitively, the c-distance between s_i and s_j is the number of times that s_i and s_j would wind up in different partitions of CBDs. The smaller the c-distance, the more similar are the constraints of the scenarios.

Definition 19. *Let $\mathcal{S}$ be an order-closed set of scenarios. The* c-ranking *of $\mathcal{S}$, denoted by $r_c(\mathcal{S})$, is the average of all pairwise c-distances between its members.*

Definition 20. *Let $\mathcal{S}$ be an order-closed set of scenarios and $D = (\mathcal{S}_1, \mathcal{S}_2)$ be a CBD of $\mathcal{S}$. The* c-ranking *of D, denoted by $r_c(D)$, is given by $r_c(\mathcal{S}_1) + r_c(\mathcal{S}_2)$.*

Example 7. Consider the set $\mathcal{S} = \{s_1, s_2, s_3\}$, where $s_1 = (acdb, \{\tau_{d,b} \geq 2\})$, $s_2 = (adbc, \{\tau_{d,c} \geq 2\})$ and $s_3 = (adcb, \{\tau_{d,b} \geq 2\})$. The set $\mathcal{S}$ is order-closed, but not realisable by a DTS. The distance functions of its members are $\mathcal{DF}_{s_1} = \{(a, b, 2, \infty), (c, b, 2, \infty), (d, b, 2, \infty)\}$, $\mathcal{DF}_{s_2} = \{(a, c, 2, \infty), (d, c, 2, \infty)\}$ and $\mathcal{DF}_{s_3} = \{(a, b, 2, \infty), (d, b, 2, \infty)\}$.

The partial order in $\mathcal{S}$, $\prec_{\mathcal{S}}$, is represented by $a{-}d{-}b$ and $a{-}c$, while $\mathcal{DF}_{\mathcal{S}}$ has only default intervals. The projections of $\mathcal{DR}_{\mathcal{S}}$ are $p_1 = (acdb, \emptyset)$, $p_2 = (adbc, \emptyset)$ and $p_3 = (adcb, \emptyset)$. So none of the members of $\mathcal{S}$ are compatible with $\mathcal{DR}_{\mathcal{S}}$.

The constraint-based disagreement between members of $\mathcal{S}$ and $\mathcal{DR}_{\mathcal{S}}$ are $dsg_{\mathcal{S}}(s_1) = \{(a, b), (c, b), (d, b)\}$, $dsg_{\mathcal{S}}(s_2) = \{(a, c), (d, c)\}$ and $dsg_{\mathcal{S}}(s_3) = \{(a, b), (d, b)\}$, while $cbd_{\mathcal{S}}(a, b) = cbd_{\mathcal{S}}(d, b) = (\{s_1, s_3\}, \{s_2\})$, $cbd_{\mathcal{S}}(a, c) = cbd_{\mathcal{S}}(d, c) = (\{s_2\}, \{s_1, s_3\})$ and $cbd_{\mathcal{S}}(c, b) = (\{s_1\}, \{s_2, s_3\})$.
This analysis leads to divisions $D_1 = (\{s_2\}, \{s_1, s_3\})$ and $D_2 = (\{s_1\}, \{s_2, s_3\})$, which have equal o-rankings and K-rankings, so we compare their c-rankings: $r_c(\{s_2\}) = 0$, $r_c(\{s_1, s_3\}) = d_c(s_1, s_3) = |\{cbd_{\mathcal{S}}(c, b)\}| = 1$, $r_c(\{s_1\}) = 0$ and $r_c(\{s_2, s_3\}) = d_c(s_2, s_3) = |\{cbd_{\mathcal{S}}(a, b), cbd_{\mathcal{S}}(a, c), cbd_{\mathcal{S}}(d, b), cbd_{\mathcal{S}}(d, c)\}| = 4$. So $r_c(D_1) = 0 + 1 = 1$ and $r_c(D_2) = 0 + 4 = 4$.

In D_1, $\{s_1, s_3\}$ is realisable by the DTS $\xi_1 || \xi_2$, where $\xi_1 = (acb, \{\tau_{a,b} \geq 2\})$ and $\xi_2 = (adb, \{\tau_{d,b} \geq 2\})$. So $\mathcal{S}$ is realised by $(\xi_1 || \xi_2) | s_2$. If we choose D_2 (which has the higher c-ranking), even though $\{s_2, s_3\}$ is order-closed, its members would not be compatible with the distance relation of the set. So we would have to divide the set once more and end up with the trivial expression $s_1 | s_2 | s_3$.

As shown above, if none of the members of an order-closed set, $\mathcal{S}$, are compatible with $\mathcal{DR}_{\mathcal{S}}$, then we compute all the CBDs of $\mathcal{S}$ and choose the one with the lowest o-ranking. We break a tie by choosing the CBD with the lowest K-ranking. If K-rankings cannot break a tie, we choose a division with the lowest c-ranking, and if necessary we use l-rankings to break a tie.

Theorem 2. *A finite set, $\mathcal{S}$, of sequential scenarios is realisable by a single DTS iff it is order-closed and every member of it is compatible with $\mathcal{DR}_{\mathcal{S}}$.*

We now present an algorithm that solves the problem of realisability of a set of scenarios as a scenario expression.

4.3 The Algorithm

Our algorithm is implemented as function *realise*, which takes a set of normalised scenarios, $\mathcal{S}$. The sequences of events in all members of $\mathcal{S}$ contain the same events, but each sequence is unique. The function iteratively performs the following:

```
Function realise(a set of scenarios S)
  if |S| = 1 then
    E := s, where S = {s};    /* trivially realisable by DTS s */
  else if ¬order-closed(S) then
    (S1, S2) := partition-order(S);
    E := realise(S1)|realise(S2);
  else /* S is order-closed */
    C := compatible(S);              /* compatible members of S */
    if C = S then
      E := realise-DTS(S);           /* Thm 2: a DTS realises S */
    else
      if C ≠ ∅ ∧ C ≠ S then
        E := realise(C)|realise(S \ C);
      else
        (S1, S2) := partition-const(S);
        E := realise(S1)|realise(S2);
  return E;
```

- If $\mathcal{S} = \{s\}$, i.e., a singleton, then *realise* returns the expression s.
- If $\mathcal{S}$ is not a singleton and is not order-closed, then *realise* invokes function *partition-order*, which returns the best OBD. Then *realise* is recursively invoked on each partition of the division.
- If $\mathcal{S}$ is order-closed, and
  - all its members are compatible with $\mathcal{DR}_{\mathcal{S}}$, then $\mathcal{S}$ is realisable by a DTS (Theorem 2). In that case, *realise-DTS* [8] is invoked, which returns the DTS that realises $\mathcal{S}$.
  - some, but not all of its members are compatible with $\mathcal{DR}_{\mathcal{S}}$, then $\mathcal{S}$ is divided into two partitions of compatible and not compatible scenarios. Then *realise* is recursively invoked on each partition.
  - none of its members are compatible with $\mathcal{DR}_{\mathcal{S}}$, then function *partition-const* is called, which returns the best CBD (by Thm. 1 there is such a CBD). Then *realise* is recursively invoked on each partition of the CBD.

To avoid additional clutter we do not show the details of dealing with gaps (see Sec. 4, p. 9). Before every invocation of *realise* the argument is partitioned, if necessary, into sets with no gaps: *realise* is then called on each of these partitions separately, and the results are combined in a choice expression.

The function *order-closed* checks whether a set of scenarios is order-closed.

In function *partition-order* variable D keeps track of the best OBD found so far. It is a five-tuple, $(\mathcal{S}_1, \mathcal{S}_2, r_o, r_k, r_l)$: the items are the two partitions, the o-ranking, the K-ranking and the l-ranking, respectively. In function *partition-const* the variable D, which is a six-tuple $(\mathcal{S}_1, \mathcal{S}_2, r_o, r_k, r_c, r_l)$, keeps track of the best CBD found so far. The additional item is the c-ranking. Given a tuple D, func-

Function partition-order(a set of scenarios $\mathcal{S}$)

$\textit{not-ordered} := \{(e, e') \in \Sigma_{\mathcal{S}} \mid e \not\prec_{\mathcal{S}} e' \wedge e' \not\prec_{\mathcal{S}} e\}$;
$D := (\emptyset, \emptyset, \infty, \infty, \infty)$; `/* best OBD, so far */`
foreach $(e, e') \in \textit{not-ordered}$ **do**
  $\mathcal{S}_1 := \{s \in \mathcal{S} \mid e \prec_s e'\}$;
  $\mathcal{S}_2 := \{s \in \mathcal{S} \mid e' \prec_s e\}$;
  $\textit{k-obd} := g_k\textit{-rank}(\mathcal{S}_1) * |\mathcal{S}_1|/|\mathcal{S}| + g_k\textit{-rank}(\mathcal{S}_2) * |\mathcal{S}_2|/|\mathcal{S}|$;
  if $\textit{better-ok}(\mathcal{S}_1, \mathcal{S}_2, D, \textit{k-obd}) \vee \textit{better-l}(\mathcal{S}_1, \mathcal{S}_2, D, \textit{k-obd})$ **then**
    $D := (\mathcal{S}_1, \mathcal{S}_2, \textit{o-rk}(\mathcal{S}_1, \mathcal{S}_2), \textit{k-obd}, \textit{l-rk}(\mathcal{S}_1, \mathcal{S}_2))$;
return $(p_1(D), p_2(D))$;

Function compatible(a set of scenarios $\mathcal{S}$)

$C := \emptyset$; `/* the set of compatible scenarios in` $\mathcal{S}$ `*/`
foreach $sc \in \mathcal{S}$ **do**
  if $\mathcal{DF}_{sc} = \mathcal{DF}_p$, *where* $p \in \mathcal{P}_{\mathcal{S}}$ *and* sc *matches* p **then**
    $C := C \cup \{sc\}$;
return C;

Function better-ok(set $\mathcal{S}_1$, set $\mathcal{S}_2$, division D, int k)

return $\textit{o-rk}(\mathcal{S}_1, \mathcal{S}_2) < \textit{o-rk}(p_1(D), p_2(D)) \vee$
$(\textit{o-rk}(\mathcal{S}_1, \mathcal{S}_2) = \textit{o-rk}(p_1(D), p_2(D)) \wedge k < \textit{K-rk}(p_1(D), p_2(D)))$;

Function better-l(set $\mathcal{S}_1$, set $\mathcal{S}_2$, division D, int k)

return $\textit{o-rk}(\mathcal{S}_1, \mathcal{S}_2) = \textit{o-rk}(p_1(D), p_2(D)) \wedge k = \textit{K-rk}(p_1(D), p_2(D)) \wedge$
$\textit{l-rk}(\mathcal{S}_1, \mathcal{S}_2) < \textit{l-rk}(p_1(D), p_2(D))$;

Function better-c(set $\mathcal{S}_1$, set $\mathcal{S}_2$, division D, int k, int c)

return $\textit{o-rk}(\mathcal{S}_1, \mathcal{S}_2) = \textit{o-rk}(p_1(D), p_2(D)) \wedge k = \textit{K-rk}(p_1(D), p_2(D)) \wedge$
$c < \textit{c-rk}(p_1(D), p_2(D))$;

Function better-cl(set $\mathcal{S}_1$, set $\mathcal{S}_2$, division D, int k, int c)

return $\textit{o-rk}(\mathcal{S}_1, \mathcal{S}_2) = \textit{o-rk}(p_1(D), p_2(D)) \wedge k = \textit{K-rk}(p_1(D), p_2(D)) \wedge$
$c = \textit{c-rk}(p_1(D), p_2(D)) \wedge \textit{l-rk}(\mathcal{S}_1, \mathcal{S}_2) < \textit{l-rk}(p_1(D), p_2(D))$;

tions p_1 and p_2 return $\mathcal{S}_1$ and $\mathcal{S}_2$. Functions *o-rk*, *K-rk*, *l-rk* and *c-rk* return the o-ranking, K-ranking, c-ranking and l-ranking of $(\mathcal{S}_1, \mathcal{S}_2)$; g_k*-rank* and g_c*-rank* are the K-ranking and the c-ranking of a partition.

Function partition-const(a set of scenarios $\mathcal{S}$)

```
/* invoked when |S| > 1 and none of the members of S is
   compatible with DR_S. */
ordered := {(e, e') ∈ Σ_S | e ≺_s e' for some s ∈ S};
CBDs := CDGs := ∅;
foreach sc ∈ S do
    CDGs := CDGs ∪ dsg(sc);
foreach (e, e') ∈ CDGs do
    S_1 := {s ∈ S | e ≺_s e' ∧ DF_s(e, e') ≠ DF_p(e, e')(s matches p ∈ P_S)};
    S_2 := S \ S_1;
    if S_2 ≠ ∅ then /* S_1 ≠ ∅ */
        CBDs := CBDs ∪ {{S_1, S_2}};
foreach (e, e') ∈ ordered \ CDGs do
    if there exists sc ∈ S, such that (e ≺_s e' ∧ DF_s(e, e') ≠ [0, ∞]) then
        S_1 := {s ∈ S | e ≺_s e'};
        S_2 := S \ S_1;
        CBDs := CBDs ∪ {{S_1, S_2}};
D := (∅, ∅, ∞, ∞, ∞, ∞);                    /* best CBD, so far */
foreach {S_1, S_2} ∈ CBDs do
    k-cbd := g_k-rank(S_1) * |S_1|/|S| + g_k-rank(S_2) * |S_2|/|S|;
    c-cbd := g_c-rank(S_1) + g_c-rank(S_2);
    if better-ok(S_1, S_2, D, k-cbd) ∨ better-c(S_1, S_2, D, k-cbd, c-cbd) ∨
      better-cl(S_1, S_2, D, k-cbd, c-cbd) then
        D := (S_1, S_2, o-rk(S_1, S_2), k-cbd, c-cbd, l-rk(S_1, S_2));
return (p_1(D), p_2(D));
```

Let N be the number of scenarios in $\mathcal{S}$ and L be the length of each scenario in $\mathcal{S}$. In the worst case the number of divisions constructed by the algorithm is $O(N)$. For each partition of a division the worst-case time complexity of constructing the distance relation is $O(N * L^2)$ (for the construction of the partial order) plus $O(L^3)$ (for the stabilisation of the distance function). The overall cost of creating the projections of the distance relation is $O(N * L^3)$. Checking the compatibility of each member of the partition with the distance relation of the partition costs $O(N * L^2)$. So the cost of treating one partition is $O(N * L^3)$. Since there are at most $O(N)$ divisions, the overall worst-case time complexity of the algorithm is $O(N^2 * L^3)$.

4.4 Revisiting an Assumption

In Sec. 4 we assumed that every member of the set of realised scenarios has a different sequence of events. In this section we discuss how to relax that assumption. Our method is best explained by an example.

Assume that we have already performed the initial partitioning (see p. 8), and that our current task is to realise the set of scenarios S, such that, for every $\xi, \eta \in S$, $\Sigma_\xi = \Sigma_\eta$. Moreover, $S = S_A \cup S_B \cup S'$, where

- $S_A = \{A_1, A_2, A_3\}$ and $eseq(A_1) = eseq(A_2) = eseq(A_3)$;
- $S_B = \{B_1, B_2\}$ and $eseq(B_1) = eseq(B_2) \neq eseq(A_1)$;
- every member of S' has a sequence of events that is unique in S.

Each projection of a DTS has a unique sequence of events, so the expression that realises S must have at least three alternatives: one for each member of S_A. However, it is possible that, say, A_2 and B_1 are projections of the same DTS. The question is how to pair the right A with the right B.

A way to address this question is to investigate all the combinations of members from S_A and S_B. More concretely, we create six sets of scenarios: $S_1 = \{A_1, B_1\} \cup S'$, $S_2 = \{A_1, B_2\} \cup S'$, $S_3 = \{A_2, B_1\} \cup S'$ etc. In each of these six sets every scenario has a unique sequence of events, so we can run our realisation algorithm separately for S_1, S_2 etc., to obtain six scenario expressions. From each of those scenario expressions we remove alternatives that realise only members of S', then find the best of the remaining expressions, i.e., one with the fewest alternatives. Let us say that it is an expression whose only alternative realises $\{A_3, B_2, D, F\}$ (where $D, F \in S'$). We retain this expression as a partial solution to the problem of realising S, and then repeat the process for $S \setminus \{A_3, B_2, D, F\}$ to find the other alternatives of the solution for S.

The problem with this approach is that the number of combinations to be investigated grows exponentially with the number of scenarios that have identical sequences. But as long as the number of combinations is not prohibitively large, the approach is quite effective.

5 Experimental Results

In Table 2 we show some experimental results for our realisation algorithm.

We have performed experiments for sets of scenarios with three different kinds of sizes, shown in the first column. For each size we considered three categories of scenarios according to their length, shown in the top row. For each kind of size and length we tried 5 different examples. Table 2 shows the minimum, maximum and average number of alternatives of the expressions generated for that set of examples, as well as the approximate running time of our entire program.

We compared these results with those from a program that performs a breadth-first search for an optimal solution (i.e., one with the minimum number of alternatives and as well-balanced as possible): in some of the cells those results are shown to the right of the vertical line. We were unable to do this for all the examples: for larger data the complete search becomes prohibitively long.

In two examples our algorithm yields 8 and 11 alternatives as opposed to the optimal 6 and 10 (see the second row of column one and column two).

Our experimental results confirm that our approach is very effective in identifying the divisions that lead to satisfactory realisations of sets of timed scenarios.

$\lvert\mathcal{S}\rvert$	$5 \leq \lvert sc\rvert \leq 10$		$10 < \lvert sc\rvert \leq 15$		$15 < \lvert sc\rvert$	
≤ 10	2, 0.2 s	2, 10.0 s	2, 0.2 s	2, 0.5 s	3, 3 s	3, 284.8 s
	6, 0.4 s	6, 51.4 s	8, 1.8 s	8, 278.2 s	7, 5 s	7, 52.3 s
	4.2, 0.3 s	4.2, 23.6 s	5.2, 1.1 s	5.2, 65.4 s	4.8, 3.8 s	4.8, 171.1 s
≤ 20	3, 0.4 s	3, 1287.2 s	3, 2.0 s	3, 7401.7 s	2, 2.6 s	2, 17866.5 s
	8, 0.8 s	6, 1477.0 s	11, 2.9 s	10, 147597.7 s	10, 10.8 s	10, 251865.8 s
	4.8, 0.5 s	4.4, 4703.8 s	5.4, 2.2 s	5.2, 32438.6 s	6.8, 15.1 s	6.8, 92812 s
> 20	3, 0.9 s		3, 2 s		3, 4.4 s	
	10, 1.6 s		12, 4.8 s		17, 642.9 s	
	5.8, 1.2 s		6.6, 3.2 s		8.25, 247.6 s	

Table 2: Experimental results

6 Conclusions

We introduced the notion of timed scenario expressions (expressions for short) and their semantics. A distributed timed scenario (DTS) is an expression. Expressions are also built by applying three operators, $|$, $\circ$ and $||$, to expressions. The operator $|$ specifies a choice between two alternatives, while $\circ$ and $||$ indicate the concatenation and the parallel composition of two expressions. We defined $[\![E]\!]$, the semantics of expression E, in terms of the set of behaviours that are specified by E, and defined the consistency of E in terms of its semantics.

We tackled the problem of realisability of sets of (sequential) scenarios as scenario expressions. We developed an algorithm that, given a set of scenarios, $\mathcal{S}$, produces an expression E that realises $\mathcal{S}$, i.e, $[\![\mathcal{S}]\!] = [\![E]\!]$.

The set $\mathcal{S}$ can sometimes be realised by a single DTS. But, in general, realising $\mathcal{S}$ involves dividing it into partitions, in such a way that each partition can be realised by a single DTS. The choice expression whose alternatives are these distributed scenarios would realise $\mathcal{S}$.

We investigated the fundamental reasons when $\mathcal{S}$ is not realisble by a DTS. This investigation resulted in developing two methods for effectively dividing $\mathcal{S}$. One is based on reasoning about the underlying partial order in $\mathcal{S}$, and the other is based on the time constraints of $\mathcal{S}$. We identified the necessary and sufficient conditions for realisability of a set of scenarios as a single DTS (Theorem 2), and thereby solved the problem that was left open in our earlier work [8].

To illustrate the effectiveness of our approach we compared our algorithm with one that performs a complete breadth-first search to find an optimal solution, i.e., one with the minimum number of alternatives. Our experimental results confirm the effectiveness and efficiency of our algorithm.

As our future work we plan to incorporate the Kleene star operator in scenario expressions, and to explore the usefulness of machine learning for solving the problem of realisability.

References

1. A. Bejleri, N. Yoshida, Synchronous multiparty session types, Electronic Notes in Theoretical Computer Science 241 (2009) 3–33, proceedings of the First Workshop on Programming Language Approaches to Concurrency and Communication-centric Software (PLACES 2008).
2. G. Castagna, M. Dezani-Ciancaglini, L. Padovani, On global types and multi-party sessions, in: R. Bruni, J. Dingel (Eds.), Formal Techniques for Distributed Systems, Springer, Berlin, Heidelberg, 2011, pp. 1–28.
3. K. Honda, N. Yoshida, M. Carbone, Multiparty asynchronous session types, J. ACM 63 (1) (2016) 9:1–9:67.
4. H. Hüttel, I. Lanese, V. T. Vasconcelos, L. Caires, M. Carbone, P. Deniélou, D. Mostrous, L. Padovani, A. Ravara, E. Tuosto, H. T. Vieira, G. Zavattaro, Foundations of session types and behavioural contracts, ACM Comput. Surv. 49 (1) (2016) 3:1–3:36.
5. F. Barbanera, I. Lanese, E. Tuosto, Formal choreographic languages, in: M. H. ter Beek, M. Sirjani (Eds.), Coordination Models and Languages, Springer Nature Switzerland, Cham, 2022, pp. 121–139.
6. M. H. ter Beek, R. Hennicker, J. Proença, Realisability of global models of interaction, in: E. Ábrahám, C. Dubslaff, S. L. T. Tarifa (Eds.), Theoretical Aspects of Computing – ICTAC 2023, Springer Nature Switzerland, Cham, 2023, pp. 236–255.
7. N. Saeedloei, F. Kluźniak, Timed scenarios: Consistency, equivalence and optimization, in: T. Massoni, M. R. Mousavi (Eds.), Formal Methods: Foundations and Applications - 21st Brazilian Symposium, SBMF 2018, Salvador, Brazil, November 26-30, 2018, Proceedings, Vol. 11254 of Lecture Notes in Computer Science, Springer, 2018, pp. 215–233.
8. N. Saeedloei, F. Kluźniak, Distributed timed scenarios, in: F. Damiani, M. Farrell (Eds.), Integrated Formal Methods - 20th International Conference, iFM 2025, Paris, France, November 19-21, 2025, Proceedings, Vol. 16194 of Lecture Notes in Computer Science, Springer, 2025, pp. 487–508. doi:10.1007/978-3-032-10794-7_24.
9. R. Dechter, I. Meiri, J. Pearl, Temporal constraint networks, Artificial Intelligence 49 (1) (1991) 61–95.
10. D. Harel, P. S. Thiagarajan, Message Sequence Charts, Springer US, Boston, MA, 2003, pp. 77–105.
11. ITU-T recommendation Z.120. Message Sequence Charts (MSC'96), May 1996, ITU Telecommunication Standardization Sector.
12. S. Balaguer, T. Chatain, S. Haar, A Concurrency-Preserving Translation from Time Petri Nets to Networks of Timed Automata, Formal Methods in System Design 40 (3) (2012) 330–355.
13. R. Alur, K. Etessami, M. Yannakakis, Inference of message sequence charts, IEEE Transactions on Software Engineering 29 (7) (2003) 623–633.
14. R. Guanciale, E. Tuosto, Realisability of pomsets, Journal of Logical and Algebraic Methods in Programming 108 (2019) 69–89. doi:https://doi.org/10.1016/j.jlamp.2019.06.003.
15. M. H. ter Beek, R. Hennicker, J. Proença, Team automata: Overview and roadmap, in: I. Castellani, F. Tiezzi (Eds.), Coordination Models and Languages, Springer Nature Switzerland, Cham, 2024, pp. 161–198.
16. R. Alur, D. L. Dill, A theory of timed automata, Theor. Comput. Sci. 126 (2) (1994) 183–235.
17. M. G. Kendall, A new measure of rank correlation, Biometrika 30 (1938) 81–93.